

Introduction to SQL

Oracle Database Programming

Prepared By
Pramod Tripathi

Learning Outcomes

After completing this module, students will be able to

- Explain the purpose of SQL.
- Identify different categories of SQL commands.
- Use Oracle SQL data types.
- Create database objects.
- Manipulate database records.
- Retrieve information using SELECT statements.
- Understand Oracle SQL environment.

What is SQL?

SQL (Structured Query Language) is the standard language used to communicate with Relational Database Management Systems (RDBMS).

SQL is used to

- Create databases and tables
- Store data
- Retrieve information
- Modify records
- Delete records
- Manage user permissions

Standard: ANSI / ISO SQL

History of SQL

- 1970 – Dr. Edgar F. Codd proposed the Relational Model.
- 1974 – IBM developed SEQUEL.
- Later renamed as SQL.
- ANSI Standard in 1986.
- ISO Standard in 1987.
- Today SQL is supported by almost every DBMS.

Popular SQL Databases

Oracle • MySQL • SQL Server • PostgreSQL • SQLite • MariaDB

Why Learn SQL?

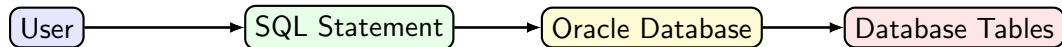
Applications

- Banking
- Railway Reservation
- Hospital Management
- E-Commerce
- Social Media
- University ERP

Advantages

- Easy to learn
- Standard Language
- High Performance
- Portable
- Secure
- Powerful Query Language

Oracle Database Environment



User writes SQL commands → Oracle processes them → Results are returned.

Categories of SQL Commands

Category	Purpose
DDL	Create and modify database objects
DML	Insert, Update and Delete records
DQL	Retrieve data using SELECT
DCL	Control user privileges
TCL	Transaction management

Mnemonic

- DDL = Structure
- DML = Data
- DQL = Query
- DCL = Security
- TCL = Transaction

Oracle SQL Data Types

Datatype	Purpose
NUMBER(p,s)	Numeric values
VARCHAR2(n)	Variable length text
CHAR(n)	Fixed length text
DATE	Date values
TIMESTAMP	Date and Time
CLOB	Large Character Data
BLOB	Binary Objects
FLOAT	Floating point numbers
RAW	Binary data

Oracle Data Types Example

```
CREATE TABLE Student(  
  
RollNo      NUMBER(5),  
  
Name        VARCHAR2(40),  
  
Gender      CHAR(1),  
  
DOB         DATE,  
  
Percentage  NUMBER(5,2),  
  
Resume      CLOB,  
  
Photo       BLOB  
  
);
```

VARCHAR2 vs CHAR

CHAR	VARCHAR2
Fixed Length	Variable Length
Consumes Full Size	Uses Required Space
Faster Comparison	Better Storage
Suitable for Gender	Suitable for Name
Suitable for State Code	Suitable for Address

Oracle Recommendation

Always use **VARCHAR2** unless the data has fixed length.

Data Definition Language (DDL)

Data Definition Language (DDL) consists of commands used to create and modify the structure of database objects.

DDL Commands

Command	Purpose
CREATE	Creates database objects
ALTER	Modifies existing objects
TRUNCATE	Removes all rows from a table
DROP	Deletes database objects permanently
RENAME	Changes the name of an object
COMMENT	Adds comments to database objects

Oracle Note

DDL statements perform an implicit COMMIT before and after execution, and they cannot be rolled back.

CREATE TABLE Command

Purpose

Creates a new table in the database.

Syntax

```
CREATE TABLE table_name  
(column1 datatype,column2 datatype,...);
```

Example

```
CREATE TABLE Student  
(  
  RollNo NUMBER,  
  Name VARCHAR2(30),  
  City VARCHAR2(25)  
);
```

CREATE TABLE with Constraints

```
CREATE TABLE Student(RollNo NUMBER PRIMARY KEY,Name VARCHAR2(40)
NOT NULL,Email VARCHAR2(60) UNIQUE,Gender CHAR(1) CHECK(Gender IN('M','F')),Fees
NUMBER(8,2) DEFAULT 0,DOB DATE);
```

Constraints

- PRIMARY KEY
- NOT NULL
- UNIQUE
- CHECK
- DEFAULT

CREATE TABLE with Foreign Key

Department Table

```
CREATE TABLE Department (DeptID NUMBER PRIMARY KEY, DeptName VARCHAR2(30));
```

Employee Table

```
CREATE TABLE Employee  
(  
    EmpID NUMBER PRIMARY KEY,  
    Name VARCHAR2(30),  
    DeptID NUMBER,  
    CONSTRAINT fk_dept  
    FOREIGN KEY (DeptID)  
    REFERENCES Department (DeptID)  
);
```

ALTER TABLE Command

Purpose

Modify the structure of an existing table.

Common Operations

- ADD Column
- MODIFY Column
- DROP Column
- RENAME Column
- ADD Constraint
- DROP Constraint

ALTER TABLE Variations

Add Column

```
ALTER TABLE Student  
ADD Mobile VARCHAR2(15);
```

Modify Datatype

```
ALTER TABLE Student  
MODIFY Name VARCHAR2(50);
```

Rename Column

```
ALTER TABLE Student  
RENAME COLUMN  
Mobile TO Phone;
```

Drop Column

```
ALTER TABLE Student  
DROP COLUMN Phone;
```

Adding and Dropping Constraints

Add Primary Key

```
ALTER TABLE Student  
ADD CONSTRAINT pk_student  
PRIMARY KEY(RollNo);
```

Add Check Constraint

```
ALTER TABLE Student  
ADD CONSTRAINT chk_fee  
CHECK(Fees>=0);
```

Drop Constraint

```
ALTER TABLE Student  
DROP CONSTRAINT chk_fee;
```

TRUNCATE TABLE

Purpose

Deletes every row from a table while preserving its structure.

```
TRUNCATE TABLE Student;
```

Characteristics

- Removes all records
- Structure remains
- Very fast
- Cannot use WHERE clause
- Implicit COMMIT

DROP TABLE

Purpose

Deletes the table permanently.

```
DROP TABLE Student;
```

Oracle Variations

```
DROP TABLE Student  
CASCADE CONSTRAINTS;
```

```
DROP TABLE Student  
PURGE;
```

PURGE bypasses Oracle Recycle Bin.

DELETE vs TRUNCATE vs DROP

Feature	DELETE	TRUNCATE	DROP
Removes Data	Yes	Yes	Yes
Removes Structure	No	No	Yes
WHERE Allowed	Yes	No	No
Rollback	Yes*	No	No
Speed	Slow	Fast	Fastest
Storage Released	No	Partial	Yes

*Rollback is possible only before COMMIT.

Data Manipulation Language (DML)

DML commands are used to manipulate (add, modify and remove) records stored inside database tables.

DML Commands

Command	Purpose
INSERT	Add new records
UPDATE	Modify existing records
DELETE	Remove records
MERGE	Insert or Update (Oracle)

Unlike DDL commands, DML commands can be rolled back before COMMIT.

INSERT Statement I

Purpose

Adds one or more records into a table.

Syntax

```
INSERT INTO table_name  
VALUES(value1,value2,...);
```

Example

```
INSERT INTO Student VALUES  
(101,'Rahul','M','2006-01-15',82.50);
```

INSERT Using Column Names

Specifying column names is considered a good programming practice.

```
INSERT INTO Student(RollNo,Name,Percentage) VALUES(102,'Amit',91.60);
```

Advantages

- Order of columns can change.
- Easier to read.
- Prevents insertion errors.

Different INSERT Variations

Insert NULL Values

```
INSERT INTO Student  
VALUES  
(103,  
'Neha',  
NULL,  
DATE '2005-08-14',  
NULL);
```

Insert Current Date

```
INSERT INTO Student VALUES(104,'Riya','F',SYSDATE,88.60);
```

INSERT ALL (Oracle)

Oracle supports inserting data into multiple tables simultaneously.

```
INSERT ALL
```

```
  INTO Science
```

```
VALUES(101,'Raj')
```

```
  INTO Commerce
```

```
VALUES(102,'Amit')
```

```
SELECT * FROM DUAL;
```

Useful for

- Data migration
- Data warehousing
- ETL applications

INSERT Using SELECT

Copies records from another table.

```
INSERT INTO Student_Backup  
  
SELECT *  
  
FROM Student;
```

Copy selected records

```
INSERT INTO MeritStudents  
  
SELECT *  
  
FROM Student  
  
WHERE Percentage >= 80;
```

UPDATE Statement

Updates existing records.

Syntax

```
UPDATE table_name  
  
SET column=value  
  
WHERE condition;
```

Example

```
UPDATE Student  
  
SET Percentage=95  
  
WHERE RollNo=101;
```

UPDATE Variations

Update multiple columns

```
UPDATE Student  
  
SET  
  
Name='Rahul Sharma',  
  
Percentage=92.50  
  
WHERE RollNo=101;
```

Increase all percentages

```
UPDATE Student  
  
SET Percentage=Percentage+5;
```

DELETE Statement

Deletes records from a table.

Delete one record

```
DELETE FROM Student  
  
WHERE RollNo=101;
```

Delete multiple records

```
DELETE FROM Student  
  
WHERE Percentage<35;
```

Delete all rows

```
DELETE FROM Student;
```

MERGE Statement (Oracle)

MERGE performs INSERT and UPDATE together.

```
MERGE INTO Student S
USING NewStudent N
ON(S.RollNo=N.RollNo)
WHEN MATCHED THEN
UPDATE SET
S.Name=N.Name
WHEN NOT MATCHED THEN
INSERT VALUES(N.RollNo,N.Name);
```

Useful for synchronization.

Common DML Errors

Error	Reason
ORA-00001	Duplicate Primary Key
ORA-01400	NULL inserted into NOT NULL column
ORA-00947	Missing values
ORA-01722	Invalid Number
ORA-02291	Parent Key Not Found

Tip

Always verify the table structure using
`DESC Student;`
before inserting records.

Data Query Language (DQL)

DQL (Data Query Language) is used to retrieve information from one or more tables.

Only one command belongs to DQL.

SELECT

Using SELECT, we can

- Display all records
- Display selected columns
- Filter records
- Sort records
- Generate reports

SELECT Statement

General Syntax

```
SELECT column_list  
FROM table_name;
```

Examples

```
SELECT *  
FROM Student;
```

```
SELECT RollNo, Name  
FROM Student;
```

Selecting Specific Columns

Display Name and Percentage.

```
SELECT Name,  
        Percentage  
FROM Student;
```

Display Name, City and Fees.

```
SELECT Name,  
        City,  
        Fees  
FROM Student;
```

Only requested columns are displayed.

Column Aliases

Aliases improve readability.

```
SELECT Name AS Student_Name,  
       Percentage AS Marks  
FROM Student;
```

Oracle also allows

```
SELECT Name Student_Name,  
       Percentage Marks  
FROM Student;
```

Arithmetic Expressions

Arithmetic operators may be used in SELECT.

```
SELECT Name,  
       Fees,  
       Fees+500 Bonus,  
       Fees*12 AnnualFees  
FROM Student;
```

Supported operators

+ - * /

Oracle DUAL Table

DUAL is a special one-row one-column table.

```
SELECT SYSDATE  
FROM DUAL;
```

```
SELECT USER  
FROM DUAL;
```

```
SELECT 25*10  
FROM DUAL;
```

Useful for testing expressions.

WHERE Clause

WHERE filters records.

```
SELECT *  
FROM Student  
WHERE Percentage>80;
```

Another example

```
SELECT Name  
FROM Student  
WHERE Gender='F';
```

Comparison Operators

Operator	Meaning
=	Equal
≠	Not Equal
>	Greater Than
<	Less Than
≥	Greater Than or Equal
≤	Less Than or Equal

```
SELECT *  
FROM Student  
WHERE Percentage>=75;
```

Logical Operators

AND

```
SELECT *  
FROM Student  
WHERE Gender='M'  
AND Percentage>80;
```

OR

```
SELECT *  
FROM Student  
WHERE Gender='F'  
OR Percentage>90;
```

NOT Operator and Practice

NOT

```
SELECT *  
FROM Student  
WHERE NOT Gender='M';
```

Another example

```
SELECT *  
FROM Student  
WHERE NOT Percentage<35;
```

Privilege Commands (DCL)

Data Control Language (DCL) is used to control access to database objects.

Oracle provides two DCL commands:

Command	Purpose
GRANT	Gives privileges
REVOKE	Removes privileges

Privileges can be granted to

- Users
- Roles
- PUBLIC

GRANT Command

General Syntax

```
GRANT privilege  
ON table_name  
TO user_name;
```

Example

```
GRANT SELECT  
ON Student  
TO SCOTT;
```

SCOTT can now read records from the Student table.

GRANT Variations

Grant multiple privileges

```
GRANT SELECT,  
INSERT,  
UPDATE
```

```
ON Student
```

```
TO SCOTT;
```

Grant all privileges

```
GRANT ALL  
ON Student  
TO SCOTT;
```

REVOKE Command

Syntax

```
REVOKE privilege  
  
ON table_name  
  
FROM user_name;
```

Example

```
REVOKE INSERT  
  
ON Student  
  
FROM SCOTT;
```

SCOTT can no longer insert records.

DESCRIBE Command

DESCRIBE displays the structure of a table.

Syntax

```
DESC Student;
```

or

```
DESCRIBE Student;
```

Sample Output

Column	Datatype
RollNo	NUMBER
Name	VARCHAR2
Gender	CHAR
DOB	DATE
Percentage	NUMBER

DISTINCT Keyword

DISTINCT removes duplicate values.
Without DISTINCT

```
SELECT City  
FROM Student;
```

With DISTINCT

```
SELECT DISTINCT City  
FROM Student;
```

Result

Ahmedabad

Surat

Rajkot

Vadodara

ORDER BY Clause

ORDER BY sorts records.

Ascending Order

```
SELECT *  
  
FROM Student  
  
ORDER BY Name;
```

Descending Order

```
SELECT *  
  
FROM Student  
  
ORDER BY Percentage DESC;
```

GROUP BY Clause

GROUP BY creates groups for aggregate functions.

Example

```
SELECT City,  
  
COUNT(*)  
  
FROM Student  
  
GROUP BY City;
```

Possible Aggregate Functions

- COUNT()
- SUM()
- AVG()
- MAX()
- MIN()

HAVING Clause

HAVING filters groups after GROUP BY.

Example

```
SELECT City,  
  
AVG(Percentage)  
  
FROM Student  
  
GROUP BY City  
  
HAVING AVG(Percentage)>80;
```

Difference

- WHERE filters rows.
- HAVING filters groups.

Complete Oracle SQL Example

```
SELECT City,  
COUNT(*) TotalStudents,  
AVG(Percentage) AverageMarks  
FROM Student  
WHERE Percentage>=35  
GROUP BY City  
HAVING COUNT(*)>=2  
ORDER BY AverageMarks DESC;
```

This query demonstrates

- WHERE
- GROUP BY
- HAVING
- ORDER BY
- Aggregate Functions

SQL Views

Definition

A View is a virtual table whose contents are derived from one or more base tables.

Unlike a table, a view does not store data permanently. It stores only the SQL query.

Advantages

- Provides data security
- Simplifies complex SQL queries
- Restricts access to selected columns
- Provides logical data independence
- Improves code reusability

CREATE VIEW

Syntax

```
CREATE VIEW view_name AS SELECT column_list FROM table_name WHERE condition;
```

Oracle Example

```
CREATE VIEW IT_Students AS SELECT RollNo,Name,Percentage  
FROM Student  
WHERE Department='IT';
```

Using a View

Once a view is created, it behaves like a table.

Display all records.

```
SELECT *  
  
FROM IT_Students;
```

Display selected columns.

```
SELECT Name,  
        Percentage  
  
FROM IT_Students;
```

Observation

Users do not need to know the original query used to create the view.

Updating a View

If the view is based on a single table and satisfies Oracle's update rules, it can be updated.

Example

```
UPDATE IT_Students  
  
SET Percentage=92  
  
WHERE RollNo=101;
```

Delete using View

```
DELETE FROM IT_Students  
  
WHERE RollNo=105;
```

Changes are reflected in the original table.

CREATE OR REPLACE VIEW

The OR REPLACE option modifies an existing view without dropping it.

```
CREATE OR REPLACE VIEW IT_Students AS
```

```
SELECT RollNo,  
       Name,  
       City,  
       Percentage  
FROM Student  
WHERE Department='IT';
```

Advantages

- No need to DROP the view.
- Existing users continue to use the view.
- Simplifies maintenance.

DROP VIEW

The DROP VIEW statement removes a view from the database.

Syntax

```
DROP VIEW view_name;
```

Example

```
DROP VIEW IT_Students;
```

Important Points

- Only the view is deleted.
- Base table data remains unchanged.
- The view can be recreated using CREATE VIEW.

WITH READ ONLY View

A read-only view allows users to retrieve data but prevents any modification.

Example

```
CREATE VIEW MeritStudents AS  
  
SELECT RollNo,  
       Name,  
       Percentage  
  
FROM Student  
  
WHERE Percentage >= 80  
  
WITH READ ONLY;
```

Attempting INSERT, UPDATE or DELETE through this view results in an Oracle error.

WITH CHECK OPTION

WITH CHECK OPTION ensures that INSERT or UPDATE operations satisfy the condition used in the view.

Example

```
CREATE VIEW IT_Students AS  
SELECT *  
FROM Student  
WHERE Department='IT'  
WITH CHECK OPTION;
```

Allowed

```
UPDATE IT_Students SET Department='IT' WHERE RollNo=101;
```

Not Allowed

```
UPDATE IT_Students SET Department='CE' WHERE RollNo=101;
```

Advantages and Limitations of Views

Advantages

- Data Security
- Simplifies Queries
- Hides Complex Joins
- Logical Data Independence
- Reusable SQL
- Restricts Column Access

Limitations

- Some views are not updateable.
- Performance depends on base tables.
- Cannot always insert data.
- Complex views may be slower.
- Base table must exist.

Introduction to SQL Functions

A SQL function is a predefined program that performs a specific operation on data and returns a result.

Functions are used to

- Manipulate character data
- Perform mathematical calculations
- Format dates
- Convert data types
- Handle NULL values
- Calculate summary statistics

Oracle provides more than 100 built-in SQL functions.

Classification of SQL Functions

Function Type	Examples
Character	UPPER, LOWER, LENGTH, SUBSTR
Numeric	ROUND, MOD, ABS, CEIL
Date	SYSDATE, ADD_MONTHS
Conversion	TO_CHAR, TO_DATE
NULL	NVL, NVL2, COALESCE
Aggregate	COUNT, SUM, AVG, MAX, MIN

Character Functions : UPPER(), LOWER(), INITCAP()

These functions change the case of characters.

UPPER()

```
SELECT UPPER('oracle')  
FROM DUAL;
```

Output

ORACLE

LOWER()

```
SELECT LOWER('DATABASE')  
FROM DUAL;
```

Output

database

INITCAP() Function

INITCAP converts the first letter of every word into uppercase.

```
SELECT INITCAP('oracle database')  
  
FROM DUAL;
```

Output

Oracle Database

Practical Example

```
SELECT Name,  
INITCAP(Name)  
  
FROM Student;
```

Useful for displaying names in proper format.

LENGTH() and SUBSTR()

LENGTH()

Returns the number of characters.

```
SELECT LENGTH('DATABASE')
```

```
FROM DUAL;
```

Output

8

SUBSTR()

Extracts a portion of a string.

```
SELECT SUBSTR('DATABASE',1,4)
```

```
FROM DUAL;
```

Output

DATA

INSTR() Function

Purpose

INSTR() returns the position of a character or substring within a string.

Syntax

```
INSTR(string, substring)
```

Example 1

```
SELECT INSTR('DATABASE', 'BASE')  
FROM DUAL;
```

Output

5

Example 2

```
SELECT INSTR('ORACLE SQL', 'SQL') FROM DUAL;
```

Output

8

REPLACE() Function

REPLACE() substitutes one substring with another.

Syntax

```
REPLACE(string,  
        search,  
        replacement)
```

Example

```
SELECT REPLACE('Oracle SQL',  
              'SQL',  
              'Database')  
  
FROM DUAL;
```

Output

Oracle Database

Useful for data cleaning and standardization.

TRIM(), LTRIM() and RTRIM()

These functions remove unwanted spaces or characters.

TRIM()

```
SELECT TRIM(' Oracle ')FROM DUAL;
```

Output

Oracle

LTRIM()

```
SELECT LTRIM('*****Oracle','*')FROM DUAL;
```

Output

Oracle

RTRIM()

```
SELECT RTRIM('Oracle*****','*')FROM DUAL;
```

Output

Oracle

LPAD() and RPAD()

LPAD() pads characters on the left.

RPAD() pads characters on the right.

LPAD()

```
SELECT LPAD('SQL',8,'*')  
FROM DUAL;
```

Output

*****SQL

RPAD()

```
SELECT RPAD('SQL',8,'*')  
FROM DUAL;
```

Output

SQL*****

CONCAT() Function

CONCAT() joins two strings.

Syntax

```
CONCAT(string1,string2)
```

Example

```
SELECT CONCAT('Oracle', ' SQL')FROM DUAL;
```

Output

Oracle SQL

Oracle also supports the concatenation operator.

```
SELECT 'Oracle' || ' SQL'FROM DUAL;
```

Output

Oracle SQL

Numeric Functions

Numeric functions perform mathematical calculations.

Function	Purpose
ROUND()	Rounds a number
TRUNC()	Truncates decimal places
MOD()	Returns remainder
ABS()	Absolute value
CEIL()	Smallest integer greater than number
FLOOR()	Largest integer less than number
POWER()	Raises a number to a power
SQRT()	Square root

Examples of Numeric Functions

ROUND()

```
SELECT ROUND(45.678,2)
FROM DUAL;
```

Output: 45.68

TRUNC()

```
SELECT TRUNC(45.678,2)
FROM DUAL;
```

Output: 45.67

MOD()

```
SELECT MOD(25,4)
FROM DUAL;
```

Output: 1

ABS(), POWER(), SQRT(), CEIL(), FLOOR()

```
SELECT ABS(-25)
FROM DUAL;
```

Output: 25

```
SELECT POWER(5,3)
FROM DUAL;
```

Output: 125

```
SELECT SQRT(144)
FROM DUAL;
```

Output: 12

```
SELECT CEIL(45.2), FLOOR(45.8)
FROM DUAL;
```

Output: 46 45

Date Functions

Oracle stores dates in DATE datatype.

Function	Purpose
SYSDATE	Current system date
CURRENT_DATE	Current session date
ADD_MONTHS()	Add months
MONTHS_BETWEEN()	Difference in months
NEXT_DAY()	Next specified weekday
LAST_DAY()	Last day of month

Examples of Date Functions

```
SELECT SYSDATE  
FROM DUAL;
```

```
SELECT ADD_MONTHS  
(SYSDATE,6)  
FROM DUAL;
```

```
SELECT LAST_DAY  
(SYSDATE)  
FROM DUAL;
```

```
SELECT NEXT_DAY  
(SYSDATE, 'MONDAY')  
FROM DUAL;
```

Examples of Date Functions

Assume today's date is: 03-AUG-2026

SQL Statement	Output
SELECT SYSDATE FROM DUAL;	03-AUG-2026
SELECT CURRENT_DATE FROM DUAL;	03-AUG-2026
SELECT ADD_MONTHS(SYSDATE,6) FROM DUAL;	03-FEB-2027
SELECT MONTHS_BETWEEN(DATE '2026-12-31', DATE '2026-08-03') FROM DUAL;	4.90
SELECT NEXT_DAY(SYSDATE,'MONDAY') FROM DUAL;	10-AUG-2026
SELECT LAST_DAY(SYSDATE) FROM DUAL;	31-AUG-2026

Observation

- **SYSDATE** returns the current database server date and time.
- **CURRENT_DATE** returns the current session date.
- **ADD_MONTHS()** adds or subtracts months.
- **MONTHS_BETWEEN()** returns the difference in months.
- **NEXT_DAY()** finds the next specified weekday.
- **LAST_DAY()** returns the last day of the month.

SQL Operators

SQL Operators are special symbols or keywords used to perform operations on data.

Oracle SQL operators are classified as

- ① Arithmetic Operators
- ② Comparison (Relational) Operators
- ③ Logical Operators
- ④ Special Operators
- ⑤ Set Operators

Arithmetic Operators

Arithmetic operators perform mathematical calculations.

Operator	Description
+	Addition
-	Subtraction
*	Multiplication
/	Division

Example

```
SELECT 20+10 Addition,20-10 Subtraction,20*10 Multiplication,20/10 Division FROM  
DUAL;
```

Addition	Subtraction	Multiplication	Division
30	10	200	2

Comparison (Relational) Operators

Comparison operators compare two values.

Operator	Meaning
=	Equal
≠	Not Equal
>	Greater Than
<	Less Than
≥	Greater Than or Equal
≤	Less Than or Equal

Example

```
SELECT *  
  
FROM Student  
  
WHERE Percentage>=80;
```

Logical Operators

Logical operators combine multiple conditions.

AND

```
SELECT * FROM Student WHERE Department='IT' AND Percentage>80;
```

OR

```
SELECT * FROM Student WHERE Department='IT' OR Department='CE';
```

NOT

```
SELECT * FROM Student WHERE NOT Gender='F';
```

BETWEEN Operator

The **BETWEEN** operator selects values within a specified range.

Syntax

```
SELECT column_name  
FROM table_name  
WHERE column_name  
BETWEEN value1 AND value2;
```

Example

```
SELECT RollNo,  
       Name,  
       Percentage  
FROM Student  
WHERE Percentage  
BETWEEN 70 AND 90;
```

Returns students whose percentage lies between 70 and 90 (inclusive).

IN and NOT IN Operators

The **IN** operator checks multiple values.

Example

```
SELECT * FROM Student WHERE Department IN ('IT','CE');
```

NOT IN Example

```
SELECT *  
  
FROM Student WHERE Department NOT IN ('IT','CE');
```

Useful when comparing against multiple values.

LIKE Operator and Wildcards

LIKE is used for pattern matching.

Wildcard	Meaning
%	Zero or more characters
-	Exactly one character

Names starting with A

```
SELECT *  
FROM Student  
WHERE Name LIKE 'A%';
```

Names ending with a

```
SELECT *  
FROM Student  
WHERE Name LIKE '%a';
```

Pattern Matching Examples

Second character is 'A'

```
SELECT * FROM Student WHERE Name LIKE '_A%';
```

Exactly four characters

```
SELECT * FROM Student WHERE Name LIKE '____';
```

Contains "RA"

```
SELECT * FROM Student WHERE Name LIKE '%RA%';
```

Observation

- % matches any number of characters.
- _ matches exactly one character.

IS NULL and EXISTS

IS NULL

Returns rows containing NULL values.

```
SELECT * FROM Student WHERE Mobile IS NULL;
```

IS NOT NULL

```
SELECT * FROM Student WHERE Mobile IS NOT NULL;
```

EXISTS

Returns TRUE if the subquery returns at least one row.

```
SELECT * FROM Department D WHERE EXISTS  
(SELECT * FROM Student S WHERE S.DeptNo=D.DeptNo);
```

Set Operators

Set Operators combine the results of two or more SELECT statements.

Operator	Purpose
UNION	Combines results and removes duplicate rows
UNION ALL	Combines all rows including duplicates
INTERSECT	Returns only common rows
MINUS	Returns rows from the first query that are not in the second query

Rules

- Both SELECT statements must have the same number of columns.
- Corresponding columns should have compatible data types.
- ORDER BY is used only once at the end.

UNION and UNION ALL

UNION

Removes duplicate rows.

```
SELECT Name  
FROM IT_Students  
UNION  
SELECT Name  
FROM CE_Students;
```

UNION ALL

Keeps duplicate rows.

```
SELECT Name  
FROM IT_Students  
UNION ALL  
SELECT Name FROM CE_Students;
```

INTERSECT and MINUS

INTERSECT

Returns only common rows.

```
SELECT Name  
FROM IT_Students  
INTERSECT  
SELECT Name  
FROM Sports_Students;
```

MINUS

Returns rows present in the first query but not in the second.

```
SELECT Name FROM IT_Students MINUS SELECT Name FROM Sports_Students;
```

Operator Precedence in Oracle SQL

When multiple operators are used together, Oracle evaluates them in the following order:

Priority	Operator
1	Arithmetic Operators (*, /, +, -)
2	Comparison Operators (=, <>, >, <, ≥, ≤)
3	IS NULL, LIKE, IN, BETWEEN
4	NOT
5	AND
6	OR

Recommendation

Use parentheses to improve readability and avoid ambiguity.

Introduction to SQL Joins

A JOIN combines rows from two or more tables based on a related column.

Example Tables

STUDENT

RollNo	Name	DeptID
101	Rahul	10
102	Neha	20
103	Amit	10
104	Riya	40

DEPARTMENT

DeptID	Department
10	IT
20	CE
30	ME

INNER JOIN

Returns only matching rows from both tables.

```
SELECT S.RollNo,  
       S.Name,  
       D.Department  
  
FROM Student S  
  
INNER JOIN Department D  
  
ON S.DeptID=D.DeptID;
```

Output

101	Rahul	IT
102	Neha	CE
103	Amit	IT

LEFT OUTER JOIN

Returns all rows from the left table (Student) and the matching rows from the right table (Department).

Query

```
SELECT S.RollNo, S.Name, D.Department FROM Student S LEFT OUTER JOIN Department D
      ON S.DeptID = D.DeptID;
```

Output

RollNo	Name	Department
101	Rahul	IT
102	Neha	CE
103	Amit	IT
104	Riya	NULL

Observation

All students are displayed. Since DeptID 40 does not exist, Department is shown as **NULL**.

RIGHT OUTER JOIN

Returns all rows from the right table (Department) and matching rows from the left table (Student).

Query

```
SELECT S.RollNo,S.Name,D.Department FROM Student S RIGHT OUTER JOIN Department D  
ON S.DeptID = D.DeptID;
```

Output

RollNo	Name	Department
101	Rahul	IT
103	Amit	IT
102	Neha	CE
NULL	NULL	ME

Observation

Department ****ME**** has no students, so the student columns are ****NULL****.

FULL OUTER JOIN

Returns all matching and non-matching rows from both tables.

Query

```
SELECT S.RollNo,S.Name,D.Department FROM Student S  
FULL OUTER JOIN Department D ON S.DeptID = D.DeptID;
```

Output

RollNo	Name	Department
101	Rahul	IT
103	Amit	IT
102	Neha	CE
104	Riya	NULL
NULL	NULL	ME

CROSS JOIN (Cartesian Product)

A **CROSS JOIN** returns the Cartesian Product of two tables. Every row from the first table is combined with every row from the second table.

Sample Tables

Student

RollNo	Name
101	Rahul
102	Neha
103	Amit

Department

Department
IT
CE
ME

Query

```
SELECT RollNo,  
       Name,  
       Department  
FROM Student  
CROSS JOIN Department;
```

CROSS JOIN Output

Output

RollNo	Name	Department
101	Rahul	IT
101	Rahul	CE
101	Rahul	ME
102	Neha	IT
102	Neha	CE
102	Neha	ME
103	Amit	IT
103	Amit	CE
103	Amit	ME

Observation

- Student table has 3 rows.
- Department table has 3 rows.
- Result = $3 \times 3 = 9$ rows.
- No matching condition is used.

NATURAL JOIN

A **NATURAL JOIN** automatically joins tables based on columns having the same name and compatible data types.

Sample Tables

Student

RollNo	Name	DeptID
101	Rahul	10
102	Neha	20
103	Amit	10
104	Riya	40

Department

DeptID	Department
10	IT
20	CE
30	ME

Query

```
SELECT * FROM Student NATURAL JOIN Department;
```

NATURAL JOIN Output

Output

RollNo	Name	Department
101	Rahul	IT
102	Neha	CE
103	Amit	IT

Observation

- Oracle automatically joins using the common column **DeptID**.
- No **ON** clause is required.
- Student **Riya** is not displayed because DeptID = 40 has no matching department.
- Department **ME** is not displayed because no student belongs to DeptID = 30.

CROSS JOIN vs NATURAL JOIN

Feature	CROSS JOIN	NATURAL JOIN
Condition	Not Required	Automatically uses common column
Result	Cartesian Product	Only Matching Rows
Number of Rows	$m \times n$	Depends on matching values
ON Clause	Not Used	Not Required
Purpose	Generate all combinations	Combine related tables

Remember

- CROSS JOIN = Every row with every row.
- NATURAL JOIN = Automatic INNER JOIN using common column names.

SQL Constraints

A constraint is a rule applied to a table column that restricts the type of data that can be stored.

Objectives

- Maintain data accuracy
- Prevent invalid data entry
- Maintain database consistency
- Enforce business rules
- Preserve data integrity

Constraints can be defined during CREATE TABLE or added later using ALTER TABLE.

Types of SQL Constraints

Constraint	Purpose
NOT NULL	Column cannot contain NULL values
CHECK	Validates data using a condition
UNIQUE	Prevents duplicate values
PRIMARY KEY	Uniquely identifies each row
FOREIGN KEY	Maintains referential integrity

These constraints maintain different types of data integrity.

Need for Constraints

Without constraints, a database may contain

- Duplicate records
- Missing values
- Invalid values
- Broken relationships
- Inconsistent information

Example

Student Percentage = 150

Student Gender = X

Department ID = 500 (does not exist)

These problems can be prevented using constraints.

NOT NULL Constraint

NOT NULL prevents a column from storing NULL values.

Example

```
CREATE TABLE Student(RollNo NUMBER,Name VARCHAR2(30)NOT NULL, City VARCHAR2(20));
```

Invalid Statement

```
INSERT INTO Student  
VALUES(101,NULL,'Ahmedabad');
```

Oracle returns an error because Name cannot be NULL.

CHECK Constraint

CHECK validates values before inserting or updating.

Example

```
CREATE TABLE Student(RollNo NUMBER,Gender CHAR(1) CHECK(Gender  
    IN('M','F')),Percentage NUMBER CHECK(Percentage>=0 AND Percentage<=100));
```

Allowed Values

Gender = M or F

Percentage = 0 to 100

UNIQUE Constraint

The **UNIQUE** constraint ensures that duplicate values are not allowed in a column.

Syntax

```
CREATE TABLE Student(RollNo NUMBER,Email VARCHAR2(50)UNIQUE);
```

Example

```
INSERT INTO Student VALUES(101,'rahul@gmail.com');  
INSERT INTO Student VALUES(102,'rahul@gmail.com');
```

The second INSERT generates an Oracle error because the Email value already exists.

UNIQUE Constraint Example

Student Table

RollNo	Email
101	rahul@gmail.com
102	neha@gmail.com
103	amit@gmail.com

Attempt to Insert

104	rahul@gmail.com
-----	-----------------

Result

ORA-00001: unique constraint violated

PRIMARY KEY Constraint

A **PRIMARY KEY** uniquely identifies every record in a table.

Characteristics

- Unique values only
- NULL values are not allowed
- Only one PRIMARY KEY per table

Example

```
CREATE TABLE Student
(  
  RollNo NUMBER  
  PRIMARY KEY,  
  
  Name VARCHAR2(30),  
  
  City VARCHAR2(20)  
);
```

Composite Primary Key

A Composite Primary Key consists of two or more columns.

Example

```
CREATE TABLE Result(RollNo NUMBER,SubjectCode VARCHAR2(10),Marks  
    NUMBER,CONSTRAINT pk_result PRIMARY KEY(RollNo,SubjectCode));
```

Each student can have only one record for a subject.

UNIQUE vs PRIMARY KEY

Feature	UNIQUE	PRIMARY KEY
Duplicate Values	Not Allowed	Not Allowed
NULL Values	Allowed (Oracle permits one or more NULLs)	Not Allowed
Number per Table	Multiple	Only One
Purpose	Candidate Key	Main Identifier
Can be Referenced by Foreign Key	Yes (if appropriately defined)	Yes

Interview Question

Every Primary Key is Unique, but every Unique key is not a Primary Key.

Referential Integrity

Referential Integrity ensures that relationships between tables remain consistent.

Foreign Key

- A Foreign Key is a column in the child table.
- It refers to the Primary Key of the parent table.
- It prevents invalid references.

Example

Department (Parent)



Student (Child)

Creating Parent and Child Tables

Parent Table

```
CREATE TABLE Department(DeptID NUMBER PRIMARY KEY,DeptName VARCHAR2(30));
```

Child Table

```
CREATE TABLE Student(RollNo NUMBER PRIMARY KEY,Name VARCHAR2(30),  
DeptID NUMBER,CONSTRAINT fk_dept FOREIGN KEY(DeptID)REFERENCES  
Department(DeptID));
```

Foreign Key Example with Output

Insert into Parent Table

```
INSERT INTO Department VALUES(10,'IT');
```

```
INSERT INTO Department VALUES(20,'CE');
```

Valid Insert

```
INSERT INTO Student VALUES(101,'Rahul',10);
```

Output

Record inserted successfully.

Invalid Insert

```
INSERT INTO Student VALUES (102,'Neha',50);
```

Output

ORA-02291: integrity constraint violated
(parent key not found)

ON DELETE CASCADE and SET NULL

ON DELETE CASCADE

Deleting a parent record automatically deletes all related child records.

```
CREATE TABLE Student(RollNo NUMBER,DeptID NUMBER,FOREIGN KEY(DeptID)  
REFERENCES Department(DeptID) ON DELETE CASCADE);
```

ON DELETE SET NULL

```
FOREIGN KEY(DeptID)REFERENCES Department(DeptID) ON DELETE SET NULL
```

Deletes the parent row and sets the child foreign key to NULL.

Summary of SQL Constraints

Constraint	Purpose
NOT NULL	Does not allow NULL values
CHECK	Validates values using a condition
UNIQUE	Prevents duplicate values
PRIMARY KEY	Uniquely identifies each record
FOREIGN KEY	Maintains referential integrity

Chapter Summary

Topics Covered

- SQL Data Types
- DDL Commands
- DML Commands
- DQL (SELECT)
- Views
- SQL Functions
- SQL Operators
- DCL Commands (GRANT and REVOKE)
- JOIN
- Constraints